

Software Engineering

GazeKey

An accessible virtual keyboard enabling hands-free typing using eye tracking and a standard webcam

Hadar Ashkenazi 211602891

Nitzan Bar-Or 315467266

Introduction

As part of the Software Engineering Workshop, we are developing **GazeKey** - a virtual keyboard controlled entirely by eye movements. This project is designed to assist individuals with motor disabilities (such as ALS) by allowing them to type using only a standard webcam, replacing the need for a physical keyboard.

To meet the workshop's requirement of creating a "product with external integration abilities", our system is designed to interact directly with the Operating System. It integrates open-source libraries (OpenCV and MediaPipe) to detect gaze, and then seamlessly injects input into other running applications. This allows the user to operate real-world software- such as sending messages on WhatsApp Web or searching on Google- demonstrating a complete engineering solution that bridges computer vision with practical desktop usage.

Specification

Solution Description

GazeKey is a desktop-based Human-Computer Interaction (HCI) solution that enables hands-free text input using eye movements only, captured via a standard webcam. The system translates the user's gaze into virtual keyboard interactions, allowing text input into any active application without requiring physical hardware, mouse clicks, or touch input.

The application runs as a transparent overlay on top of the user's desktop environment. By continuously analyzing the webcam video stream, GazeKey detects eye landmarks, computes the user's gaze point on the screen, and maps it to a virtual keyboard layout. Key selection is performed using a dwell-time mechanism, enabling reliable interaction without physical switches.

The solution is fully client-side, prioritizing low latency, privacy, and real-time responsiveness.

Functional Requirements

The system provides the following core functionalities:

1. Real-Time Gaze Tracking:
The application continuously tracks the user's eye movements by extracting facial and eye landmarks from the webcam video stream.
Using these landmarks, the system computes a stable gaze vector representing the user's point of regard (PoR) on the screen.
2. Gaze-to-Key Mapping:
The detected gaze position is normalized and mapped to screen coordinates corresponding to the virtual keyboard overlay. This mapping is based on a prior calibration process, allowing the system to adapt the gaze-to-screen transformation to the individual user. Each key on the keyboard is defined by a bounding region, enabling precise hit detection when the gaze intersects a key area.
3. Dwell-Time Based Selection:
To trigger a key press, the system uses a dwell-time mechanism:
When the user fixates on a key for a configurable duration (e.g., 1.0 second), a selection event is triggered.
If the gaze moves away before the dwell time completes, the timer resets.
This approach prevents accidental activations (the "Midas Touch" problem) and removes the need for external input devices.

4. Visual Feedback:

To ensure usability and confidence, the system provides immediate visual feedback:

- Focus Indication: The key currently under gaze is visually highlighted.
- Progress Indicator: A visual timer (e.g., filling ring or bar) shows dwell-time progression toward activation.
- Activation Confirmation: A short visual or auditory cue confirms successful input.

5. External Application Integration:

Once a key is selected, the system synthesizes a native operating-system keystroke event and injects it into the active foreground application.

This enables seamless interaction with third-party software such as browsers, text editors, and messaging platforms without requiring modifications or plugins.

6. Context Control:

A dedicated pause/resume mechanism allows users to temporarily disable gaze tracking.

This prevents unintended inputs during rest periods or while observing the screen without interaction.

7. Multi-Language Support:

- Language Selection: The system supports toggling between Hebrew and English (QWERTY) keyboard layouts.
- Dynamic Layout Adjustment: The virtual keyboard interface automatically updates the key labels and symbols based on the selected language.
- Directional Adaptation: The system manages input directionality (Right-to-Left for Hebrew, Left-to-Right for English) to ensure seamless integration with external applications.

8. Predictive Text & Auto-complete:

- Dictionary-Based Suggestions: The system provides real-time word suggestions based on the characters already typed.
- Selection Mechanism: Suggested words appear in a dedicated area of the virtual keyboard, allowing the user to select a full word via dwell-time.
- Personalized Learning: The system tracks frequently used words and adjusts the suggestion priority to improve typing speed over time.

9. Calibration System:

- 5-Point Setup: The system starts with a quick setup where the user looks at 5 points on the screen (corners and center) to ensure the gaze tracking is accurate.

- Drift Detection: The system automatically identifies if the accuracy drops by noticing patterns like frequent use of the "Backspace" key in a specific area.
- Manual Recalibration: A dedicated button on the virtual keyboard allows the user to restart the calibration at any time.
- Optional - Accuracy Alert: A visual icon appears on the screen to warn the user if the system detects that the gaze is no longer aligned correctly.

Non-Functional Requirements

1. Responsiveness:
Real-time operation with minimal perceptible latency.
2. Privacy:
All video processing is performed locally; no visual data is stored or transmitted.
3. Reliability:
Stable gaze detection under minor head movement.
4. Configurability:
Adjustable dwell-time thresholds and keyboard layout preferences.

User Flow (Interaction Logic)

The operational flow of GazeKey is designed as a continuous loop that monitors the user's gaze and translates it into intentional keyboard input.

1. Frame Acquisition
The system continuously captures video frames from the user's webcam and detects the user's face and eye regions.
2. Eye Position Estimation
Based on the detected eye landmarks, the system determines the relative position of the user's iris and estimates where the user is looking on the screen.
3. Screen Mapping
The estimated gaze position is mapped to a specific (X, Y) coordinate on the screen overlay that contains the virtual keyboard.
4. Target Identification
The system checks whether the mapped screen coordinates intersect with the defined area of any virtual key.
5. Validation (Dwell-Time Logic)
 - a. If gaze remains stable: The system starts or updates an internal timer and

displays visual progress on the focused key, indicating that a selection is in progress.

b. If gaze shifts away: The timer is immediately reset, and no action is triggered, preventing accidental selections.

6. Execution

Once the dwell time threshold is reached, the system generates a standard keyboard input event and sends it to the operating system.

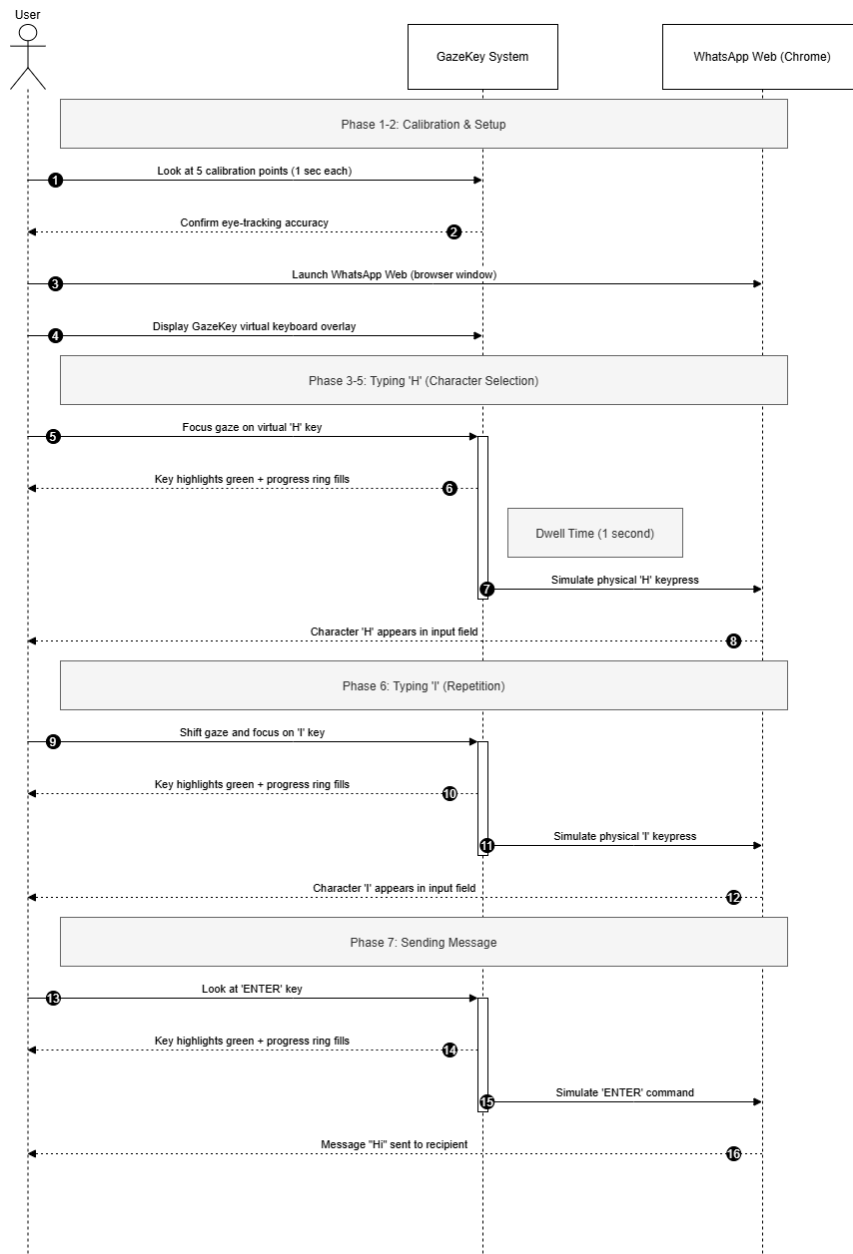
A visual or auditory cue confirms successful key activation.

Usage Scenario

1. Messaging on WhatsApp Web

Objective: The user wishes to send the message "Hi" using a web browser.

1. Calibration: The system displays 5 points on the screen (four corners and center). The user looks at each point for a second to calibrate the eye-tracking accuracy.
2. Setup: Two windows are open simultaneously on the computer screen: one for WhatsApp Web and the other for the keyboard (GazeKey).
3. Character Selection: The user focuses their eyes on the virtual 'H' key.
4. Feedback: The 'H' key highlights in green. A progress ring begins to fill.
5. Trigger: After 1 second of fixation, the system simulates a physical 'H' keypress. The character appears in the WhatsApp input field.
6. Repetition: The user shifts gaze to 'l', waits for the visual confirmation, and the character is typed.
7. Send: The user looks at the 'ENTER' key. The system triggers the enter command, sending the message to the recipient.



Sequence diagram illustrating gaze-based text input using GazeKey in WhatsApp Web.

2. Performing a Google Search

Objective: The user wishes to search for the term "Dog" using the Google search engine.

1. Calibration: The system displays 5 points on the screen (four corners and center). The user looks at each point for a second to calibrate the eye-tracking accuracy.

2. Setup: The user opens a browser to Google.com and ensures the text cursor is blinking in the search bar. The GazeKey overlay is active on the screen.
3. First Character: The user fixes their gaze on the virtual 'D' key.
4. Feedback: The 'D' key highlights in green (or the chosen theme color), and the progress indicator begins to fill.
5. Trigger: Upon completion of the dwell time (e.g., 1 second), the system injects the 'D' keystroke, and the letter appears in the Google search bar.
6. Sequence Completion: The user repeats the process for the letters 'O' and 'G' to complete the word.
7. Search Execution: The user looks at the 'ENTER' key. The system triggers the command, and the browser loads the search results page.



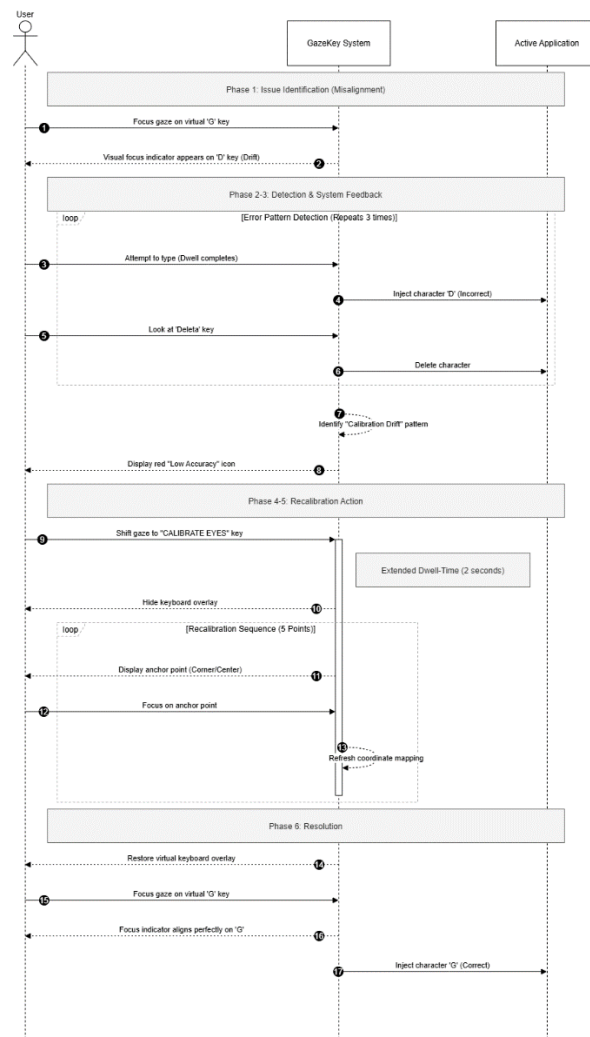
Wireframe mockup illustrating the GazeKey virtual keyboard overlay interacting with a web browser (Google Chrome) during gaze-based text input.

3. Handling Calibration Drift

Objective: The user wishes to restore typing accuracy after the system gaze detection becomes misaligned due to head movement.

1. Issue Identification: The user focuses on the virtual 'G' key, but the visual focus indicator appears on the 'D' key instead

2. Detection: The user attempts to type 'G' but the system injects 'D'. The user immediately looks at 'BACKSPACE' to delete the character and repeats this three times.
3. Feedback: The system identifies this error pattern as "Calibration Drift" and displays a red "Low Accuracy" icon in the corner of the overlay.
4. Action: The user shifts their gaze to the "RECALIBRATE" function key and maintains focus for 2 seconds (extended dwell-time).
5. Recalibration Process: The keyboard disappears, and 5 anchor points appear sequentially (corners and center). The user focuses on each point to refresh the coordinate mapping.
6. Resolution: The virtual keyboard reappears. The user focuses on the 'G' key, the focus indicator aligns perfectly, and the character is typed correctly.



Sequence diagram illustrating detection of calibration drift and user-guided recalibration in the GazeKey system.

Design

Architectural Overview

The GazeKey system operates as a local desktop application designed with a modular Pipeline Architecture. The system processes data sequentially: capturing raw video, extracting biological features, mapping logic to screen coordinates, and finally synthesizing operating system events.

Since privacy and low latency are paramount, the architecture is entirely client-side; no video data is transmitted to external servers. The system functions as a background process with a "Top-Most" transparent GUI overlay.

System Components Overview

The system is divided into two main layers:

- Processing & Logic Layer - responsible for gaze detection, interaction logic, and decision-making.
- Presentation & UI Layer - responsible for rendering the virtual keyboard and providing visual feedback.

Each layer consists of dedicated components that communicate through well-defined interfaces.

Logic & Processing Layer:

1. Video Capture Component

Responsibility:

Access the webcam and provide a continuous stream of video frames.

Description:

This component interfaces with the system's webcam using OpenCV. It captures frames in real time and forwards them to the eye detection component for further processing.

2. Eye Detection Component

Responsibility:

Detect the user's face and eyes and estimate eye position.

Description:

Using MediaPipe's facial landmark detection, this component identifies facial and eye landmarks in each frame.

Based on the detected landmarks, it determines the relative position of the user's iris within the eye region.

The output of this component is an estimated gaze position relative to the screen.

3. Gaze Mapping Component

Responsibility:

Translate eye position data into screen coordinates.

Description:

This component converts the relative eye position into normalized screen coordinates (X, Y) that correspond to the virtual keyboard overlay.

Basic stabilization techniques are applied to reduce noise caused by small involuntary eye movements.

4. Interaction Controller

Responsibility:

Determine whether the user intends to activate a virtual key and translate that action into a specific character based on the active language state.

Description:

The InteractionController checks whether the mapped gaze position intersects with the bounding area of any virtual key. It manages the dwell-time logic and tracks interaction states (idle, focus, and activation) to ensure that only intentional actions result in input. It also manages the language state (Hebrew/English) to ensure the gaze is mapped to the correct character.

In addition, this component manages the language mapping:

- It maintains a state machine for the active language (Hebrew or English).
- In English State: It maps the selected key area to Latin characters (e.g., 'A', 'B', 'C').
- In Hebrew State: It maps the same key area to Hebrew characters (e.g., 'א', 'ב', 'ג').
- It processes the "Language Toggle" key activation to switch between these layouts.

5. Virtual Keyboard UI

Responsibility:

Render the visual keyboard overlay and update key labels.

Description:

This component displays the keyboard on the screen. It supports multi-language layouts by dynamically changing the labels between Hebrew and

English and adjusting the visual direction (Right-to-Left for Hebrew) to match the selected language.

6. OS Input Adapter

Responsibility:

Send confirmed keyboard input to external applications.

Description:

Once a key activation is confirmed, this component generates a standard keyboard event at the operating system level using system input libraries. The input is sent to the currently active foreground application, allowing seamless interaction with third-party software.

7. Prediction Engine

Responsibility:

Provide real-time word suggestions and manage the dictionary learning mechanism.

Description:

This component generates suggestions based on the user's initial keystrokes.

- External Resources: It utilizes PyEnchant to interface with standard open-source dictionaries: Hspell for Hebrew and Aspell for English.
- Personalization: The engine maintains a local file that tracks the user's common vocabulary.
- Update Frequency: While the core dictionaries are static, the user-learned data is updated at the end of each session to prioritize frequent words in future suggestions.

8. Calibration & Drift Manager

Responsibility:

Maps eye movements to screen pixels and monitors accuracy.

Description:

This component uses the 5-point calibration data to create a mathematical map for the screen. It continuously checks the user's typing patterns. optional - if it detects consistent misses (Drift), it triggers a visual warning to let the user know it is time to recalibrate

Presentation & UI Layer:

1. Overlay Window Manager

Responsibility:
Manage the transparent overlay window.

Description:
This component creates and maintains a frameless, always-on-top window that displays the virtual keyboard.
It handles screen resolution changes and ensures that the overlay does not interfere with underlying applications.

2. Virtual Keyboard Renderer

Responsibility:
Render the virtual keyboard layout.

Description:
The renderer draws the keyboard keys, labels, and functional buttons such as Enter and Backspace.
It updates the visual state of each key based on information received from the InteractionController.

3. Visual Feedback Component

Responsibility:
Provide real-time feedback to the user.

Description:
This component highlights the currently focused key, displays dwell-time progress indicators, and provides confirmation feedback when a key is successfully activated.

Data Flow

1. Input: Frame $F(t)$ captured at time t .
2. Extraction: Iris vector v calculated.
3. Normalization: Vector v mapped to screen point $P(x,y)$.
4. Hit Testing: System checks if $P(x,y)$ intersects with any Key Region $R(k)$.
5. Validation: If P in $R(k)$ for time $T > Threshold$, trigger event.
6. Output: Synthesize keystroke K .

Data Storage & Configuration

The system does not require a relational database. Instead, it uses a lightweight Configuration File (JSON/YAML) to store user preferences across sessions.

Stored Data:

- Dwell Time Threshold (e.g., 0.5s - 2.0s).
- Keyboard Layout (QWERTY size and position).
- Calibration Offsets (to adjust for different camera angles).
- Active Language State: Stores the last used language (Hebrew or English) to maintain consistency across sessions.
- Language Layout Maps: Definitions of character mappings for both Hebrew and English layouts.
- User Dictionary Database: A lightweight local file (e.g., SQLite or JSON) containing word frequency and learned vocabulary.
- Learning Cache: Temporary storage for new words typed during the current session, merged into the main dictionary upon exit.
- Calibration Matrix: A persistent set of offset data used to normalize the gaze stream based on the user's last successful calibration.

External Libraries & Tools

To ensure reliability, performance, and cross-platform compatibility, GazeKey leverages several industry-standard open-source libraries. Each library was selected to handle a specific domain within the modular architecture:

1. Computer Vision & ML:

- OpenCV (opencv-python):
 - Role: Image Acquisition & Pre-processing.
 - Usage: Responsible for capturing the raw video stream, converting color spaces (RGB/BGR), and handling frame resizing to optimize processing speed before passing data to the ML model.
- Google MediaPipe (mediapipe):
 - Role: Biological Feature Extraction.
 - Usage: Utilizes the Face Mesh solution. Optimized for CPU inference, it allows for real-time detection of 468 landmarks and accurate iris tracking on standard laptops without requiring a GPU.

2. System Integration (OS Control):

- Pynput / PyAutoGUI:
 - Role: Input Injection Driver.
 - Usage: Serves as the bridge to the Operating System. Allows the Python script to control the mouse cursor and synthesize keyboard events, enabling integration with third-party apps.

3. Graphical User Interface (GUI):

- PyQt5 / PySide6 (Qt Framework):
 - Role: Transparent Overlay Engine.

- Usage: Used to create the frameless, transparent window for the virtual keyboard. Qt is chosen for its superior support for alpha-blending compared to basic libraries like Tkinter.

4. Mathematical Processing:

- NumPy:
 - Role: Vector Logic.
 - Usage: Handles efficient calculation of Euclidean distances and matrix operations needed for coordinate mapping.